

STUDENT HANDBOOK

Linux & ROS 2 Command Mastery

From your first terminal session to debugging a live robot: the essential commands, how they fit together, and hands-on labs to build real fluency.

Dr. Muhammad Kazim

Assistant Professor, University of Lahore

Target platform: Ubuntu 22.04 / 24.04 with ROS 2 Humble / Jazzy

How to Use This Handbook

Reading commands does not make you an expert; typing them does. Keep a terminal open beside this handbook and run every example. Each part ends with a lab: do it without looking back, then check. The handbook has two halves:

- **Part A: Linux** — navigation, files, permissions, processes, networking, packages, and shell productivity. ROS 2 lives on Linux, so this is the foundation.
- **Part B: ROS 2** — workspaces, colcon, nodes, topics, services, actions, parameters, launch, bags, TF2, and debugging.

Notation

Command	What it does	Example
<file>	Replace with your own value (do not type the angle brackets).	cat <file> → cat notes.txt
\$	Command run as a normal user (the prompt; do not type it).	\$ ls
sudo	Run the command with administrator rights. Use only when needed.	sudo apt update
Ctrl+C	Stop the running command (very common with ROS 2 nodes).	ros2 topic echo /odom → Ctrl+C

Tip: Every command has a manual. `man ls` opens the full manual (press **q** to quit); `ls --help` prints a short summary; `ros2 <verb> -h` does the same for ROS 2.

PART A — LINUX

1. The Filesystem and Navigation

Linux has a single tree starting at / (root). Your home directory is /home/<user>, shortened to ~. . is the current directory and .. is the parent.

Command	What it does	Example
<code>pwd</code>	Print the directory you are in.	<code>pwd</code>
<code>ls</code>	List directory contents.	<code>ls</code>
<code>ls -la</code>	List all files (including hidden dot-files) with details.	<code>ls -la ~</code>
<code>ls -lh</code>	Details with human-readable sizes (K, M, G).	<code>ls -lh /var/log</code>
<code>ls -lt</code>	Sort by modification time, newest first.	<code>ls -lt ~/ros2_ws/src</code>
<code>cd <dir></code>	Change directory.	<code>cd ~/ros2_ws/src</code>
<code>cd ..</code>	Go up one level.	<code>cd ..</code>
<code>cd ~ / cd</code>	Go to your home directory.	<code>cd</code>
<code>cd -</code>	Jump back to the previous directory.	<code>cd -</code>
<code>tree -L 2</code>	Show directory tree, 2 levels deep (sudo apt install tree).	<code>tree -L 2 ~/ros2_ws</code>

Key directories

Command	What it does	Example
<code>/home</code>	Users' personal files.	<code>/home/student</code>
<code>/opt</code>	Optional software. ROS 2 installs here.	<code>/opt/ros/humble</code>
<code>/etc</code>	System configuration files.	<code>/etc/hosts</code>
<code>/dev</code>	Devices: serial ports, cameras, joysticks.	<code>/dev/ttyUSB0</code> , <code>/dev/video0</code>
<code>/tmp</code>	Temporary files, cleared on reboot.	<code>/tmp/test.txt</code>
<code>/var/log</code>	System logs.	<code>/var/log/syslog</code>

2. Working with Files and Directories

Command	What it does	Example
<code>touch <file></code>	Create an empty file (or update its timestamp).	<code>touch notes.txt</code>
<code>mkdir <dir></code>	Create a directory.	<code>mkdir lab1</code>
<code>mkdir -p a/b/c</code>	Create nested directories in one go.	<code>mkdir -p ~/ros2_ws/src</code>
<code>cp <src> <dst></code>	Copy a file.	<code>cp robot.urdf backup.urdf</code>
<code>cp -r <src> <dst></code>	Copy a directory recursively.	<code>cp -r my_pkg my_pkg_old</code>
<code>mv <src> <dst></code>	Move or rename.	<code>mv old.py new.py</code>
<code>rm <file></code>	Delete a file (no recycle bin!).	<code>rm test.txt</code>
<code>rm -r <dir></code>	Delete a directory and everything in it.	<code>rm -r build install log</code>
<code>rm -i <file></code>	Ask before each deletion (safer).	<code>rm -i *.log</code>
<code>rmdir <dir></code>	Remove an empty directory.	<code>rmdir empty_dir</code>

Command	What it does	Example
<code>ln -s <target> <link></code>	Create a symbolic link (shortcut).	<code>ln -s ~/datasets data</code>
<code>file <file></code>	Identify what kind of file it is.	<code>file map.pgm</code>

Warning: `rm -rf` deletes without asking and cannot be undone. Never run it with `sudo` or on `/` or `~`. Check your path with `pwd` first.

Wildcards (globbing)

Command	What it does	Example
<code>*</code>	Matches any characters.	<code>ls *.py</code>
<code>?</code>	Matches exactly one character.	<code>ls img_?.png</code>
<code>[abc]</code>	Matches one of the listed characters.	<code>ls data[12].csv</code>
<code>{a,b}</code>	Expands to each option (brace expansion).	<code>mkdir -p pkg/{launch,config,urdf}</code>

3. Viewing and Editing Files

Command	What it does	Example
<code>cat <file></code>	Print a whole file.	<code>cat package.xml</code>
<code>less <file></code>	Scroll through a file (q quits, / searches).	<code>less /var/log/syslog</code>
<code>head -n 20 <file></code>	First 20 lines.	<code>head -n 20 data.csv</code>
<code>tail -n 20 <file></code>	Last 20 lines.	<code>tail -n 20 build.log</code>
<code>tail -f <file></code>	Follow a file live as it grows.	<code>tail -f ~/.ros/log/latest/launch.log</code>
<code>wc -l <file></code>	Count lines.	<code>wc -l data.csv</code>
<code>diff <a> </code>	Show differences between two files.	<code>diff old.yaml new.yaml</code>
<code>nano <file></code>	Simple terminal editor (Ctrl+O save, Ctrl+X exit).	<code>nano ~/.bashrc</code>
<code>vim <file></code>	Powerful editor (i insert, Esc, :wq save+quit).	<code>vim params.yaml</code>
<code>code .</code>	Open current folder in VS Code (if installed).	<code>code ~/ros2_ws</code>

4. Searching

Command	What it does	Example
<code>find <dir> -name</code>	Find files by name.	<code>find ~/ros2_ws -name "*.launch.py"</code>
<code>find -type d</code>	Find only directories.	<code>find . -type d -name config</code>
<code>find -mmin -30</code>	Files modified in the last 30 minutes.	<code>find . -mmin -30</code>
<code>grep <text> <file></code>	Search for text in a file.	<code>grep ERROR build.log</code>
<code>grep -rn <text> <dir></code>	Recursive search with line numbers.	<code>grep -rn "create_publisher" src/</code>
<code>grep -i</code>	Case-insensitive search.	<code>grep -i warning log.txt</code>
<code>grep -v</code>	Show lines that do NOT match.	<code>grep -v DEBUG log.txt</code>
<code>which <cmd></code>	Show which executable runs.	<code>which python3</code>
<code>locate <name></code>	Fast search via an index (sudo updatedb).	<code>locate libopencv</code>

5. Pipes, Redirection and Text Tools

The pipe `|` sends one command's output into the next command. This is how small tools combine into powerful ones.

Command	What it does	Example
<code>cmd > file</code>	Write output to a file (overwrites).	<code>ls > files.txt</code>
<code>cmd >> file</code>	Append output to a file.	<code>echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc</code>
<code>cmd 2> file</code>	Redirect errors only.	<code>colcon build 2> errors.txt</code>
<code>cmd &> file</code>	Redirect output and errors.	<code>./run.sh &> run.log</code>
<code>cmd1 cmd2</code>	Pipe output of cmd1 into cmd2.	<code>ros2 topic list grep camera</code>
<code>tee <file></code>	Show output AND save it.	<code>colcon build 2>&1 tee build.log</code>
<code>sort / uniq</code>	Sort lines; remove duplicates.	<code>cat ids.txt sort uniq -c</code>
<code>cut -d, -f2</code>	Extract column 2 of a CSV.	<code>cut -d, -f2 data.csv</code>
<code>awk '{print \$1}'</code>	Print the first field of each line.	<code>ps aux awk '{print \$2}'</code>
<code>sed 's/a/b/g'</code>	Find and replace in text.	<code>sed -i 's/humble/jazzy/g' setup.sh</code>
<code>xargs</code>	Turn input lines into arguments.	<code>find . -name "*.pyc" xargs rm</code>

6. Users, Permissions and Ownership

Each file has permissions for the **user** (owner), **group**, and **others**: **read** (4), **write** (2), **execute** (1). `-rwxr-xr--` means owner `rwx` (7), group `r-x` (5), others `r--` (4) = 754.

Command	What it does	Example
<code>chmod +x <file></code>	Make a file executable (needed for Python nodes/scripts).	<code>chmod +x my_node.py</code>
<code>chmod 755 <file></code>	Set permissions numerically.	<code>chmod 755 run.sh</code>
<code>chmod -R</code>	Apply recursively.	<code>chmod -R u+w my_pkg</code>
<code>chown user:group</code>	Change owner (needs sudo).	<code>sudo chown -R \$USER:\$USER ~/ros2_ws</code>
<code>sudo <cmd></code>	Run as administrator.	<code>sudo apt install htop</code>
<code>whoami / id</code>	Show current user and groups.	<code>id</code>
<code>groups</code>	List groups you belong to.	<code>groups</code>
<code>sudo usermod -aG</code>	Add yourself to a group (log out/in after).	<code>sudo usermod -aG dialout \$USER</code>
<code>passwd</code>	Change your password.	<code>passwd</code>

Robotics note: "Permission denied" on `/dev/ttyUSB0` or `/dev/ttyACM0` (Pixhawk, Arduino, LiDAR) means you are not in the `dialout` group. Fix it with `sudo usermod -aG dialout $USER`, then log out and back in.

7. Processes and System Monitoring

Command	What it does	Example
<code>ps aux</code>	List all running processes.	<code>ps aux grep ros</code>
<code>top / htop</code>	Live CPU and memory view (q quits).	<code>htop</code>
<code>kill <PID></code>	Ask a process to stop.	<code>kill 4321</code>
<code>kill -9 <PID></code>	Force-kill a stuck process.	<code>kill -9 4321</code>
<code>pkill -f <name></code>	Kill processes by name pattern.	<code>pkill -f gz</code>
<code>cmd &</code>	Run a command in the background.	<code>ros2 run turtlesim turtlesim_node &</code>
<code>jobs / fg / bg</code>	List, foreground, background jobs.	<code>fg %1</code>

Command	What it does	Example
Ctrl+Z	Pause the foreground process.	(then bg to resume in background)
nohup cmd &	Keep running after the terminal closes.	nohup ./logger.sh &
df -h	Disk space per filesystem.	df -h
du -sh <dir>	Size of a directory.	du -sh ~/bags
free -h	RAM usage.	free -h
uptime	Load averages and running time.	uptime
uname -a	Kernel and architecture info.	uname -a
lsb_release -a	Ubuntu version.	lsb_release -a
nvidia-smi	GPU status (NVIDIA only).	watch -n 1 nvidia-smi
watch -n 1 <cmd>	Re-run a command every second.	watch -n 1 "ros2 node list"

8. Hardware and Devices

Command	What it does	Example
lsusb	List USB devices.	lsusb
ls /dev/tty*	Find serial ports.	ls /dev/ttyUSB* /dev/ttyACM*
dmesg -w	Kernel messages live (see what happens when you plug in).	sudo dmesg -w
v4l2-ctl --list-devices	List cameras (sudo apt install v4l-utils).	v4l2-ctl --list-devices
lscpu / lsblk	CPU info; block devices (disks, SD cards).	lsblk
vcgencmd measure_temp	CPU temperature on Raspberry Pi.	vcgencmd measure_temp

9. Networking

Multi-machine ROS 2 (laptop + robot, ground station + drone) depends on the network. These commands diagnose most problems.

Command	What it does	Example
ip a	Show interfaces and IP addresses.	ip a
hostname -I	Just your IP addresses.	hostname -I
ping <host>	Test reachability.	ping 192.168.1.20
ssh user@host	Log in to a remote machine (e.g., the robot).	ssh pi@192.168.1.20
ssh -X user@host	SSH with graphical forwarding.	ssh -X pi@robot.local
scp <src> user@host:<dst>	Copy files over SSH.	scp map.yaml pi@192.168.1.20:~/maps/
rsync -avz	Efficient sync over SSH.	rsync -avz src/ pi@robot:~/ros2_ws/src/
ss -tulpn	Open ports and listening programs.	sudo ss -tulpn
curl / wget	Download from the web.	wget https://example.com/file.zip
nmcli dev wifi	List Wi-Fi networks.	nmcli dev wifi list
sudo ufw status	Firewall status (can block DDS).	sudo ufw status

10. Packages and Software

Command	What it does	Example
<code>sudo apt update</code>	Refresh the package lists.	<code>sudo apt update</code>
<code>sudo apt upgrade</code>	Upgrade installed packages.	<code>sudo apt upgrade</code>
<code>sudo apt install <pkg></code>	Install a package.	<code>sudo apt install ros-humble-turtlesim</code>
<code>sudo apt remove <pkg></code>	Remove a package.	<code>sudo apt remove htop</code>
<code>apt search <text></code>	Search available packages.	<code>apt search ros-humble-nav2</code>
<code>apt list --installed</code>	List installed packages.	<code>apt list --installed grep ros-</code>
<code>dpkg -L <pkg></code>	Where did a package put its files?	<code>dpkg -L ros-humble-turtlesim</code>
<code>pip install <pkg></code>	Install a Python package (prefer a venv).	<code>pip install numpy</code>
<code>python3 -m venv <dir></code>	Create a Python virtual environment.	<code>python3 -m venv ~/venvs/cv</code>

11. Archives and Compression

Command	What it does	Example
<code>tar -czvf out.tar.gz <dir></code>	Create a compressed archive.	<code>tar -czvf ws.tar.gz ros2_ws/src</code>
<code>tar -xzvf file.tar.gz</code>	Extract an archive.	<code>tar -xzvf ws.tar.gz</code>
<code>zip -r out.zip <dir></code>	Create a zip.	<code>zip -r lab.zip lab1</code>
<code>unzip file.zip</code>	Extract a zip.	<code>unzip models.zip -d models/</code>

12. Environment Variables and Bash Configuration

ROS 2 is configured almost entirely through environment variables. Understanding them removes most "package not found" mysteries.

Command	What it does	Example
<code>echo \$VAR</code>	Print a variable.	<code>echo \$ROS_DISTRO</code>
<code>export VAR=value</code>	Set a variable for this terminal.	<code>export ROS_DOMAIN_ID=7</code>
<code>env / printenv</code>	List all variables.	<code>printenv grep ROS</code>
<code>source <file></code>	Run a file in the current shell.	<code>source /opt/ros/humble/setup.bash</code>
<code>~/.bashrc</code>	Runs in every new terminal: put permanent setup here.	<code>nano ~/.bashrc</code>
<code>alias name='cmd'</code>	Create a shortcut.	<code>alias cb='colcon build --symlink-install'</code>
<code>echo \$PATH</code>	Directories searched for commands.	<code>echo \$PATH tr ':' '\n'</code>

Recommended ~/.bashrc additions for this course:

```
# ---- ROS 2 setup ----
source /opt/ros/humble/setup.bash          # use jazzy on Ubuntu 24.04
[ -f ~/ros2_ws/install/setup.bash ] && source ~/ros2_ws/install/setup.bash
export ROS_DOMAIN_ID=7                    # pick a unique number per team (0-101)
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
alias cb='cd ~/ros2_ws && colcon build --symlink-install'
alias sws='source ~/ros2_ws/install/setup.bash'
```

13. Shell Productivity: Work Like an Expert

Command	What it does	Example
Tab	Auto-complete commands, paths, ROS 2 topics. Press twice to list options.	<code>ros2 topic ec<Tab></code>
Up / Down	Scroll through previous commands.	
Ctrl+R	Search command history interactively.	<code>Ctrl+R colcon</code>
history	Show command history.	<code>history grep ssh</code>
!!	Repeat the last command.	<code>sudo !!</code>
Ctrl+A / Ctrl+E	Jump to start / end of the line.	
Ctrl+U / Ctrl+K	Delete to start / end of the line.	
Ctrl+L / clear	Clear the screen.	
cmd1 && cmd2	Run cmd2 only if cmd1 succeeds.	<code>colcon build && source install/setup.bash</code>
cmd1 ; cmd2	Run both regardless.	<code>cd ~ ; ls</code>
tmux	Split one terminal into panes; sessions survive SSH drops.	<code>tmux new -s robot</code>

Tip: On a robot over SSH, always work inside `tmux`. If Wi-Fi drops, your nodes keep running; reconnect with `tmux attach -t robot`. Split panes: **Ctrl+B %** (vertical), **Ctrl+B "** (horizontal), **Ctrl+B arrow** to move.

14. Git Essentials

Command	What it does	Example
git clone <url>	Download a repository.	<code>git clone https://github.com/ros/ros_tutorials.git -b humble</code>
git status	What has changed?	<code>git status</code>
git add <files>	Stage changes.	<code>git add src/my_pkg</code>
git commit -m	Save a snapshot.	<code>git commit -m "Add lidar node"</code>
git push / pull	Send to / get from the remote.	<code>git pull origin main</code>
git log --oneline	Compact history.	<code>git log --oneline -10</code>
git checkout -b	Create and switch to a branch.	<code>git checkout -b feature/pid</code>
git diff	Show unstaged changes.	<code>git diff</code>

15. Writing a Bash Script

```
#!/bin/bash
# save as start_robot.sh, then: chmod +x start_robot.sh && ./start_robot.sh
set -e                                # stop on first error
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
echo "Starting robot on domain $ROS_DOMAIN_ID ..."
ros2 launch my_robot bringup.launch.py use_sim:=${1:-false}
```

Lab A — Linux Fluency Challenge

Complete each task using only the terminal. Record the commands you used.

1. Create `~/labA/{data, scripts, logs}` in a single command.
2. Create five files `sensor_1.csv ... sensor_5.csv` in `data` with one command.
3. Write the text *time,value* into `sensor_1.csv`, then append three data lines.
4. Count the lines in all CSV files together, and find the largest file.
5. Write a script `scripts/hello.sh` that prints your name and date; make it executable and run it.
6. Find every file under `/opt/ros` whose name ends in `.launch.py` and count them.
7. Start `sleep 500` in the background, find its PID with `ps`, and kill it.
8. Save the output of `ip a` to `logs/net.txt` and extract only the lines containing *inet*.
9. Create an alias `ll` for `ls -lah` that survives opening a new terminal.
10. Compress `~/labA` into `labA.tar.gz`, delete the folder, then restore it.

Hint: Tasks 1, 2 and 6 can each be solved in one line using brace expansion, `find` and a pipe into `wc -l`.

PART B — ROS 2

16. Core Concepts in One Page

Command	What it does	Example
Node	A single program doing one job (camera driver, controller, planner).	/camera_node
Topic	Continuous, many-to-many data stream (publish / subscribe).	/cmd_vel, /scan, /image_raw
Service	Request / response, for quick one-off calls.	/reset, /spawn
Action	Long-running goal with feedback and cancel.	/navigate_to_pose
Parameter	Configurable value belonging to a node.	max_speed: 0.5
Interface	Message (.msg), service (.srv), or action (.action) definition.	geometry_msgs/msg/Twist
Launch file	Starts and configures many nodes at once.	bringup.launch.py
Workspace	Folder where you build your own packages.	~/ros2_ws
DDS / Domain ID	The middleware; nodes only see each other on the same domain.	ROS_DOMAIN_ID=7

Pattern: Every ros2 command follows the pattern `ros2 <verb> <sub-verb> [args]`. Type `ros2` then press Tab twice to see all verbs; `ros2 topic -h` lists topic sub-verbs.

17. Environment Setup and Checks

Command	What it does	Example
<code>source /opt/ros/<distro>/setup.bash</code>	Activate ROS 2 in this terminal (do in every terminal or in ~/.bashrc).	<code>source /opt/ros/humble/setup.bash</code>
<code>printenv grep ROS</code>	Check ROS variables are set.	<code>printenv grep ROS</code>
<code>echo \$ROS_DISTRO</code>	Which distribution is active.	<code>echo \$ROS_DISTRO # humble</code>
<code>export ROS_DOMAIN_ID=<n></code>	Isolate your team from others on the same network.	<code>export ROS_DOMAIN_ID=12</code>
<code>export ROS_LOCALHOST_ONLY=1</code>	Keep traffic on this computer (Humble). Jazzy uses ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST.	<code>export ROS_LOCALHOST_ONLY=1</code>
<code>ros2 doctor</code>	Check the installation and network for problems.	<code>ros2 doctor --report</code>
<code>ros2 wtf</code>	Alias of ros2 doctor.	<code>ros2 wtf</code>

18. Workspaces and Building with colcon

```
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash # 'overlay' your workspace on top of ROS 2
```

Command	What it does	Example
<code>colcon build</code>	Build all packages in the workspace (run from the workspace root, not src/).	<code>cd ~/ros2_ws && colcon build</code>
<code>--symlink-install</code>	Python/launch/config edits take effect without rebuilding.	<code>colcon build --symlink-install</code>

Command	What it does	Example
<code>--packages-select</code>	Build only named packages.	<code>colcon build --packages-select my_pkg</code>
<code>--packages-up-to</code>	Build a package and its dependencies.	<code>colcon build --packages-up-to my_robot_bringup</code>
<code>--parallel-workers N</code>	Limit parallel jobs (use 1-2 on Raspberry Pi to avoid freezes).	<code>colcon build --parallel-workers 1</code>
<code>--cmake-args</code>	Pass CMake options.	<code>colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release</code>
<code>--event-handlers console_direct+</code>	Show full compiler output live.	<code>colcon build --event-handlers console_direct+</code>
<code>colcon list</code>	List packages colcon can see.	<code>colcon list</code>
<code>colcon test</code>	Run package tests.	<code>colcon test --packages-select my_pkg</code>
<code>colcon test-result --verbose</code>	Show test results.	<code>colcon test-result --verbose</code>
<code>rosdep install</code>	Install all dependencies declared in package.xml files.	<code>rosdep install --from-paths src -y --ignore-src</code>
<code>rm -rf build install log</code>	Clean build (fixes many strange errors).	<code>cd ~/ros2_ws && rm -rf build install log</code>

Most common mistake: After every build, run `source install/setup.bash` in each terminal where you will run your nodes. Forgetting this is the #1 cause of "Package 'my_pkg' not found".

19. Packages

Command	What it does	Example
<code>ros2 pkg create --build-type ament_python</code>	Create a Python package.	<code>ros2 pkg create my_py_pkg --build-type ament_python --dependencies rclpy std_msgs</code>
<code>ros2 pkg create --build-type ament_cmake</code>	Create a C++ package.	<code>ros2 pkg create my_cpp_pkg --build-type ament_cmake --dependencies rclcpp</code>
<code>--node-name <name></code>	Also generate a hello-world node.	<code>ros2 pkg create demo --build-type ament_python --node-name talker</code>
<code>ros2 pkg list</code>	List all available packages.	<code>ros2 pkg list grep nav2</code>
<code>ros2 pkg executables <pkg></code>	List runnable programs in a package.	<code>ros2 pkg executables turtlesim</code>
<code>ros2 pkg prefix <pkg></code>	Where is the package installed?	<code>ros2 pkg prefix turtlesim</code>
<code>ros2 pkg xml <pkg></code>	Show its package.xml.	<code>ros2 pkg xml turtlesim</code>

20. Nodes

Command	What it does	Example
<code>ros2 run <pkg> <exe></code>	Start a node.	<code>ros2 run turtlesim turtlesim_node</code>
<code>ros2 node list</code>	List running nodes.	<code>ros2 node list</code>
<code>ros2 node info <node></code>	Show a node's publishers, subscribers, services, actions.	<code>ros2 node info /turtlesim</code>
<code>--ros-args -r __node:=</code>	Rename a node at startup.	<code>ros2 run turtlesim turtlesim_node --ros-args -r __node:=turtle2</code>
<code>--ros-args -r __ns:=</code>	Put a node in a namespace (multi-robot).	<code>ros2 run turtlesim turtlesim_node --ros-args -r __ns:=robot1</code>

Command	What it does	Example
<code>--ros-args -r old:=new</code>	Remap a topic name.	<code>ros2 run turtlesim turtle_teleop_key --ros-args -r /turtle1/cmd_vel:=/cmd_vel</code>
<code>--ros-args --log-level</code>	Change logging verbosity.	<code>ros2 run my_pkg node --ros-args --log-level debug</code>

21. Topics

Command	What it does	Example
<code>ros2 topic list</code>	List active topics.	<code>ros2 topic list</code>
<code>ros2 topic list -t</code>	List topics with their message types.	<code>ros2 topic list -t</code>
<code>ros2 topic echo <topic></code>	Print messages as they arrive.	<code>ros2 topic echo /turtle1/pose</code>
<code>echo --once / --field</code>	Print one message, or one field only.	<code>ros2 topic echo /odom --field pose.pose.position</code>
<code>ros2 topic info <topic></code>	Type and number of publishers/subscribers.	<code>ros2 topic info /scan</code>
<code>ros2 topic info -v</code>	Verbose, including QoS settings (debug QoS mismatches).	<code>ros2 topic info /scan -v</code>
<code>ros2 topic hz <topic></code>	Measure publishing rate.	<code>ros2 topic hz /camera/image_raw</code>
<code>ros2 topic bw <topic></code>	Measure bandwidth.	<code>ros2 topic bw /camera/image_raw</code>
<code>ros2 topic delay <topic></code>	Latency based on header timestamps.	<code>ros2 topic delay /scan</code>
<code>ros2 topic pub <topic> <type> <data></code>	Publish from the terminal (YAML syntax).	<code>ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.8}}"</code>
<code>pub --once / -r <Hz></code>	Publish once, or at a fixed rate.	<code>ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"</code>
<code>ros2 topic find <type></code>	Find topics of a given type.	<code>ros2 topic find sensor_msgs/msg/Image</code>

22. Interfaces (Messages, Services, Actions)

Command	What it does	Example
<code>ros2 interface list</code>	List all interface types.	<code>ros2 interface list grep sensor_msgs</code>
<code>ros2 interface show <type></code>	Show the fields of a type.	<code>ros2 interface show geometry_msgs/msg/Twist</code>
<code>ros2 interface proto <type></code>	Show a fill-in template (great for topic pub).	<code>ros2 interface proto geometry_msgs/msg/Twist</code>
<code>ros2 interface package <pkg></code>	Interfaces defined in a package.	<code>ros2 interface package nav_msgs</code>

23. Services

Command	What it does	Example
<code>ros2 service list -t</code>	List services with types.	<code>ros2 service list -t</code>
<code>ros2 service type <srv></code>	Show a service's type.	<code>ros2 service type /spawn</code>

Command	What it does	Example
<code>ros2 service find <type></code>	Find services of a type.	<code>ros2 service find std_srvs/srv/Empty</code>
<code>ros2 service call <srv> <type> <req></code>	Call a service.	<code>ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: 'turtle2'}"</code>
<code>(empty request)</code>	Services with no fields.	<code>ros2 service call /clear std_srvs/srv/Empty</code>

24. Actions

Command	What it does	Example
<code>ros2 action list -t</code>	List action servers with types.	<code>ros2 action list -t</code>
<code>ros2 action info <action></code>	Clients and servers of an action.	<code>ros2 action info /turtle1/rotate_absolute</code>
<code>ros2 action send_goal</code>	Send a goal.	<code>ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"</code>
<code>send_goal --feedback</code>	Send a goal and watch feedback.	<code>ros2 action send_goal --feedback /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: -1.57}"</code>

25. Parameters

Command	What it does	Example
<code>ros2 param list</code>	List parameters of all (or one) node.	<code>ros2 param list /turtlesim</code>
<code>ros2 param get <node> <p></code>	Read a value.	<code>ros2 param get /turtlesim background_r</code>
<code>ros2 param set <node> <p> <v></code>	Change a value at runtime.	<code>ros2 param set /turtlesim background_r 150</code>
<code>ros2 param describe</code>	Type, description, and constraints.	<code>ros2 param describe /turtlesim background_r</code>
<code>ros2 param dump <node></code>	Print all params as YAML (redirect to save).	<code>ros2 param dump /turtlesim > turtlesim.yaml</code>
<code>ros2 param load <node> <file></code>	Load params from YAML into a running node.	<code>ros2 param load /turtlesim turtlesim.yaml</code>
<code>--ros-args --params-file</code>	Start a node with a YAML file.	<code>ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml</code>
<code>--ros-args -p name:=value</code>	Set a single param at startup.	<code>ros2 run my_pkg ctrl --ros-args -p kp:=1.2</code>

26. Launch Files

Command	What it does	Example
<code>ros2 launch <pkg> <file></code>	Start a launch file.	<code>ros2 launch turtlesim multisim.launch.py</code>
<code>arg:=value</code>	Pass launch arguments.	<code>ros2 launch my_robot bringup.launch.py use_sim_time:=true</code>

Command	What it does	Example
<code>--show-args</code>	List a launch file's arguments.	<code>ros2 launch nav2_bringup navigation_launch.py --show-args</code>
<code>ros2 launch <path/file></code>	Run a launch file directly by path.	<code>ros2 launch ./test.launch.py</code>

Minimal Python launch file (`my_pkg/launch/demo.launch.py`):

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='turtlesim', executable='turtlesim_node', name='sim'),
        Node(package='turtlesim', executable='turtle_teleop_key',
            prefix='xterm -e', output='screen'),
    ])
```

Remember: For a Python package, launch files must be installed. Add to `setup.py` `data_files`:
`(os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py'))`.

27. Recording and Replaying Data (rosbag2)

Command	What it does	Example
<code>ros2 bag record <topics></code>	Record specific topics.	<code>ros2 bag record /scan /odom /tf</code>
<code>ros2 bag record -a</code>	Record every topic (careful: images are huge).	<code>ros2 bag record -a</code>
<code>-o <name></code>	Name the output bag.	<code>ros2 bag record -o run1 /imu/data</code>
<code>ros2 bag info <bag></code>	Duration, topics, message counts.	<code>ros2 bag info run1</code>
<code>ros2 bag play <bag></code>	Replay a bag.	<code>ros2 bag play run1</code>
<code>play -r <rate> / -l</code>	Replay faster/slower, or loop.	<code>ros2 bag play run1 -r 0.5 -l</code>
<code>play --clock</code>	Publish /clock (use with <code>use_sim_time:=true</code>).	<code>ros2 bag play run1 --clock</code>

28. TF2 (Coordinate Frames)

Command	What it does	Example
<code>ros2 run tf2_tools view_frames</code>	Generate a PDF of the whole TF tree.	<code>ros2 run tf2_tools view_frames</code>
<code>ros2 run tf2_ros tf2_echo <a> </code>	Print the transform between two frames.	<code>ros2 run tf2_ros tf2_echo map base_link</code>
<code>static_transform_publisher</code>	Publish a fixed transform (e.g., sensor mount).	<code>ros2 run tf2_ros static_transform_publisher --x 0.1 --z 0.2 --frame-id base_link --child-frame-id laser</code>
<code>ros2 topic echo /tf_static</code>	Inspect static transforms.	<code>ros2 topic echo /tf_static</code>

29. Visualization and GUI Tools

Command	What it does	Example
<code>rviz2</code>	3D visualization of robots, maps, point clouds, TF.	<code>rviz2 -d my_config.rviz</code>

Command	What it does	Example
<code>rqt</code>	Plugin-based GUI toolbox.	<code>rqt</code>
<code>rqt_graph</code>	Graph of nodes and topics: see who talks to whom.	<code>rqt_graph</code>
<code>rqt_plot / plotjuggler</code>	Plot numeric topic fields over time.	<code>ros2 run plotjuggler plotjuggler</code>
<code>rqt_image_view</code>	View camera image topics.	<code>ros2 run rqt_image_view rqt_image_view</code>
<code>rqt_console</code>	Filter and inspect log messages.	<code>ros2 run rqt_console rqt_console</code>
<code>rqt_reconfigure</code>	Change parameters with sliders.	<code>ros2 run rqt_reconfigure rqt_reconfigure</code>

30. Logs and Debugging

Command	What it does	Example
<code>~/ .ros/log/</code>	Where ROS 2 writes log files.	<code>ls -lt ~/ .ros/log head</code>
<code>ros2 topic echo /rosout</code>	All node log messages in one stream.	<code>ros2 topic echo /rosout</code>
<code>ros2 daemon stop / start</code>	Restart discovery cache (fixes stale topic/node lists).	<code>ros2 daemon stop && ros2 daemon start</code>
<code>ros2 multicast send / receive</code>	Test whether two machines can discover each other.	<code>ros2 multicast receive # on PC B</code>
<code>ros2 component list</code>	List loaded composable nodes.	<code>ros2 component list</code>
<code>ros2 lifecycle get / set</code>	Inspect and drive managed (lifecycle) nodes, e.g., Nav2.	<code>ros2 lifecycle get /map_server</code>

31. Troubleshooting Checklist

Command	What it does	Example
Package not found	Workspace not sourced or not built.	<code>colcon build && source install/setup.bash</code>
No executable found	Missing entry_points in setup.py, or install() in CMakeLists.txt.	<code>ros2 pkg executables my_pkg</code>
Topic visible but echo shows nothing	QoS mismatch (e.g., sensor data is BEST_EFFORT).	<code>ros2 topic info /scan -v</code>
Nodes can't see each other	Different ROS_DOMAIN_ID, firewall, or different networks.	<code>echo \$ROS_DOMAIN_ID ; ros2 multicast receive</code>
Others' topics appear on your screen	Shared Wi-Fi, same domain: choose a unique ID.	<code>export ROS_DOMAIN_ID=23</code>
TF 'extrapolation' errors	Clock mismatch or missing use_sim_time.	<code>--ros-args -p use_sim_time:=true</code>
Build freezes on Raspberry Pi	Out of RAM from parallel compiling.	<code>colcon build --parallel-workers 1</code>
Serial device permission denied	User not in dialout group.	<code>sudo usermod -aG dialout \$USER</code>

Lab B — ROS 2 Command Mastery with turtlesim

Install with `sudo apt install ros-humble-turtlesim ros-humble-rqt*` (use *jazzy* on Ubuntu 24.04). Use a separate terminal for each running node.

1. Set `ROS_DOMAIN_ID` to your roll number modulo 100 and make it permanent.
2. Start `turtlesim_node` and `turtle_teleop_key`. Open `rqt_graph` and explain every arrow.
3. Find the message type of `/turtle1/cmd_vel` and print its fields.
4. Make the turtle drive a circle at 1 Hz using only `ros2 topic pub`.
5. Measure the publishing rate of `/turtle1/pose`. Print only the x coordinate.
6. Spawn *turtle2* at (3, 3) with a service call, then clear the screen.
7. Change the background to blue at runtime, dump all parameters to YAML, and restart the node using that file.
8. Rotate *turtle1* to 90° with an action goal while watching feedback. Cancel a goal mid-way with Ctrl+C.
9. Run a second turtlesim in namespace `/robot2` and drive it with teleop via remapping.
10. Record 20 s of `/turtle1/cmd_vel` while driving, reset the sim, and replay the bag at 2x speed.
11. Create a Python package `lab_b` containing a launch file that starts both turtlesims from task 9. Build, source, and launch it.
12. Break it on purpose: open a new terminal without sourcing and run your launch file. Read the error and fix it.

Path to Expert: Weekly Habits

- **No GUI file manager for one week.** Do everything in the terminal.
- **Before asking for help**, run: `ros2 node list`, `ros2 topic list -t`, `ros2 topic info -v`, `rqt_graph`, and read the full error message.
- **Read source launch files** of real packages (`ros2 pkg prefix nav2_bringup`, then explore *share/*).
- **Keep a personal cheat-sheet** of every command that solved a problem for you.
- **Use Tab completion constantly.** It prevents typos in long topic and type names.

One-Page Quick Reference

Navigate	<code>pwd · ls -la · cd <dir> · cd .. · cd -</code>
Files	<code>mkdir -p · cp -r · mv · rm -r · touch · ln -s</code>
View	<code>cat · less · head · tail -f · nano</code>
Search	<code>find . -name "*.py" · grep -rn text src/</code>
Pipes	<code> · > · >> · 2>&1 · tee · xargs</code>
Permissions	<code>chmod +x · chown · sudo · usermod -aG dialout \$USER</code>
Processes	<code>ps aux · htop · kill -9 · pkill -f · Ctrl+C · Ctrl+Z · &</code>
System	<code>df -h · du -sh · free -h · lsusb · dmesg -w</code>
Network	<code>ip a · ping · ssh · scp · rsync -avz</code>
Packages	<code>sudo apt update · apt install · apt search</code>
ROS env	<code>source /opt/ros/humble/setup.bash · ROS_DOMAIN_ID · ros2 doctor</code>
Build	<code>colcon build --symlink-install · --packages-select · source install/setup.bash</code>
Deps	<code>rosdep install --from-paths src -y --ignore-src</code>
Create pkg	<code>ros2 pkg create <name> --build-type ament_python ament_cmake</code>
Nodes	<code>ros2 run · ros2 node list · ros2 node info</code>
Topics	<code>list -t · echo · info -v · hz · bw · pub</code>
Services	<code>ros2 service list -t · ros2 service call</code>
Actions	<code>ros2 action list -t · ros2 action send_goal --feedback</code>
Params	<code>ros2 param list · get · set · dump · load</code>
Interfaces	<code>ros2 interface show · ros2 interface proto</code>
Launch	<code>ros2 launch <pkg> <file> arg:=value · --show-args</code>
Bags	<code>ros2 bag record -o name · info · play -r 0.5</code>
TF	<code>view_frames · tf2_echo a b · static_transform_publisher</code>
Tools	<code>rviz2 · rqt_graph · rqt_console · plotjuggler</code>
Remap	<code>--ros-args -r old:=new · -r __ns:=/robot1 · -p key:=val</code>